

Name of Student: Bhoomi Rai	PRN:26070122513
Semester: III	Year : AY 2026-27
Subject: Programming with Java	
ASSIGNMENT No : 10	
TITLE : Abstract Classes and Methods	

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Theory

Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

Exercise

1.Create an abstract Payment class and implement Credit Card and UPI payment classes.?

Programme

```
src > assignment10 > J PaymentDemo.java > Java > {} assignment10
1 package assignment10;
2
3 abstract class Payment {
4
5     abstract void makePayment();
6 }
7
8 class CreditCard extends Payment {
9
10     void makePayment() {
11         System.out.println(x: "Payment made using Credit Card.");
12     }
13 }
14
15 class UPI extends Payment {
16
17     void makePayment() {
18         System.out.println(x: "Payment made using UPI.");
19     }
20 }
21
22 public class PaymentDemo {
23
24     Run | Debug | Run main | Debug main
25     public static void main(String[] args) {
26
27         CreditCard card = new CreditCard();
28         UPI upi = new UPI();
29
30         card.makePayment();
31         upi.makePayment();
32     }
33 }
```

Output

```
Payment made using Credit Card.
Payment made using UPI.
```

2. Create an abstract FoodOrder class with an abstract method calculateBill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.

Programme

```
ountDemo.java 2, U    J LibraryBook.java 2, U    J PaymentDemo.java 5, U    J FoodOrderDemo.java 5, U X  ▸ ▾
src > assignment10 > J FoodOrderDemo.java > Java > DineInOrder
1  package assignment10;
2
3  abstract class FoodOrder {
4
5      double amount;
6
7      FoodOrder(double amount) {
8          this.amount = amount;
9      }
10
11     abstract void calculateBill();
12 }
13
14 class DineInOrder extends FoodOrder {
15
16     DineInOrder(double amount) {
17         super(amount);
18     }
19
20     void calculateBill() {
21
22         double total = amount + 50;
23
24         System.out.println(x: "Dine-In Order");
25         System.out.println("Food Amount: " + amount);
26         System.out.println("Total Bill: " + total);
27     }
28 }
```

```
ountDemo.java 2, U    J LibraryBook.java 2, U    J PaymentDemo.java 5, U    J FoodOrderDemo.java 5, U X  ▸ ▾
src > assignment10 > J FoodOrderDemo.java > Java > DineInOrder
14 class DineInOrder extends FoodOrder {
20     void calculateBill() {
26         System.out.println("Total Bill: " + total);
27     }
28 }
29
30 class TakeAwayOrder extends FoodOrder {
31
32     TakeAwayOrder(double amount) {
33         super(amount);
34     }
35
36     void calculateBill() {
37
38         double total = amount + 30;
39
40         System.out.println(x: "Take-Away Order");
41         System.out.println("Food Amount: " + amount);
42         System.out.println("Total Bill: " + total);
43     }
44 }
45
46 public class FoodOrderDemo {
47     Run | Debug | Run main | Debug main
48     public static void main(String[] args) {
49
50         DineInOrder dineIn = new DineInOrder(amount: 500);
```

```
src > assignment10 > J FoodOrderDemo.java > Java > DineInOrder
30 class TakeAwayOrder extends FoodOrder {
36     void calculateBill() {
43     }
44 }
45
46 public class FoodOrderDemo {
47     Run | Debug | Run main | Debug main
48     public static void main(String[] args) {
49
50         DineInOrder dineIn = new DineInOrder(amount: 500);
51         TakeAwayOrder takeAway = new TakeAwayOrder(amount: 500);
52
53         dineIn.calculateBill();
54
55         System.out.println();
56
57         takeAway.calculateBill();
58     }
59 }
```

Output

```
Dine-In Order
Food Amount: 500.0
Total Bill: 550.0

Take-Away Order
Food Amount: 500.0
Total Bill: 530.0
```

Conclusion

Through these experiments, we understood the concept of abstraction using abstract classes and abstract methods in Java. We learned that an abstract class cannot be instantiated directly and can contain both abstract and concrete methods. In the Payment System, CreditCard and UPI classes implemented the abstract payment method according to their own requirements. In the Food Order System, DineInOrder and TakeAwayOrder provided their own implementations of calculateBill(). Thus, abstraction helps hide implementation details, reduce code duplication, improve flexibility, and make programs easier to maintain and extend.

GitHub

bhoomiralbtech2025-ui / Java

Search Type to search

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Java

Public

Pin

Watch

main

1 Branch

0 Tags

Go to file

Add file

Code

bhoomiralbtech2025-ui

Payment

36c8d1 · now

20 Commits

.vscode

Calculator

13 hours ago

Assesment10

Payment

now